Lab # 9

Introduction to Filters in Digital Signal Processing (DSP): FIR and IIR

1. Objective

- Understand the Design Principles: The lab aims to provide students with a solid understanding of the design principles behind FIR and IIR filters. This includes comprehending the underlying mathematical concepts, filter structures, and design methods used for both FIR and IIR filters.

2. Overview

Digital Filters:

Digital filters process digital signals to modify or extract specific information.
They can be categorized as Finite Impulse Response (FIR) filters or Infinite Impulse Response (IIR) filters. Digital filters are widely used in various applications such as audio processing, image processing, communications, and control systems.

Finite Impulse Response (FIR) Filters:

FIR filters have a finite impulse response, meaning that their output depends only on a finite number of input samples. The impulse response of an FIR filter is a finite sequence of coefficients. FIR filters are known for their linear phase response and stability characteristics.
Design methods for FIR filters include windowing, frequency sampling, and the Parks-McClellan algorithm.

Infinite Impulse Response (IIR) Filters:

IIR filters have an infinite impulse response, meaning that their output depends on both current and past input samples as well as past output samples.
IIR filters can achieve a desired frequency response using fewer coefficients compared to FIR filters. IIR filters can exhibit non-linear phase response, which can be both advantageous and challenging depending on the application. Design methods for IIR filters include Butterworth, Chebyshev, and elliptic designs.

Filter Design:

Filter design involves determining the filter coefficients that achieve a desired frequency response. In Python, various libraries such as SciPy and NumPy provide functions for designing FIR and IIR filters. Designing filters typically involves specifying parameters such as the filter type (low-pass, high-pass, etc.), cutoff frequencies, passband ripple, and stopband attenuation.

Filter Implementation:

Once the filter coefficients are obtained, they can be applied to the input signal for filtering.
In Python, the SciPy library provides functions for implementing both FIR and IIR filters.
The filtering operation involves convolving the input signal with the filter coefficients for FIR filters or using recursive equations for IIR filters.
Care should be taken to handle issues such as filter transients and potential instability in IIR filters.

Lab Task

- Compare the performance characteristics of FIR and IIR filters, such as passband ripple, stopband attenuation, and phase response.
- Investigate the trade-offs between FIR and IIR filters in terms of computational complexity, filter order, and stability.
- Discuss the advantages and disadvantages of FIR and IIR filters for different signal processing applications.

EXPERIMENT NO: 10

IMPLEMENTATIONS OF LP AND HP FIR FILTER

Objective: To implement LP FIR filter for a given sequence.

Software: Python

THEORY:

FIR filters are digital filters with finite impulse response. They are also known as **non-recursive digital filters** as they do not have the feedback.

An FIR filter has two important advantages over an IIR design:

- Firstly, there is no feedback loop in the structure of an FIR filter. Due to not having a feedback loop, an FIR filter is inherently stable. Meanwhile, for an IIR filter, we need to check the stability.
- Secondly, an FIR filter can provide a linear-phase response. As a matter of fact, a linear-phase response is the main advantage of an FIR filter over an IIR design otherwise, for the same filtering specifications; an IIR filter will lead toa lower order.

FIR FILTER DESIGN

MRCE

An FIR filter is designed by finding the coefficients and filter order that meet certain specifications, which can be in the time-domain (e.g. a matched filter) and/or the frequency domain (most common). Matched filters perform a cross-correlation between the input signal and a known pulse-shape. The FIR convolution is a cross-correlation between the input signal and a time-reversed copy of the impulse-response. Therefore, thematched-filter's impulse response is "designed" by sampling the known pulse-shape and using those samples in reverse order as the coefficients of the filter.

When a particular frequency response is desired, several different design methods are common:

1. Window design method
2. Frequency Sampling method
3. Weighted least squares design

WINDOW DESIGN METHOD

In the window design method, one first designs an ideal IIR filter and then truncates the infinite impulse response by multiplying it with a finite length window function. The result is a finite impulse response filter whose frequency response is modified from that of the IIR filter

Algorithm:

Step I : Enter the pass band frequency (fp) and stop band frequency (fq).

Step II : Get the sampling frequency (fs), length of window (n).

Step III : Calculate the cut off frequency, fn

Step IV : Use boxcar, hamming, blackman Commands to design window.

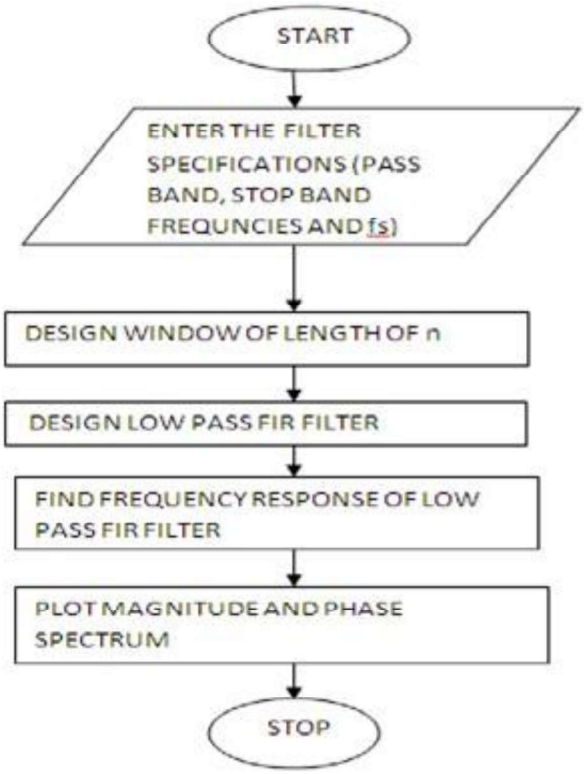
Step V : Design filter by using above parameters.

Step VI : Find frequency response of the filter using matlab command freqz.

Step VII : Plot the magnitude response and phase response of the filter.

MRCE

Flow Chart:



MRCE

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, freqz

# Input specifications
wp = 0.375 * np.pi
ws = 0.5 * np.pi
wt = ws - wp

# Calculate the order of the filter
n1 = np.ceil(8 * np.pi / wt)
N = int(n1 + (n1 - 1) % 2)

print('Order of the FIR filter N =', N)

# Calculate the filter coefficients using the Hamming window
Wc1 = wp + wt / 2

# Calculate the cutoff frequency
fs = 2 * np.pi # Assuming the sampling frequency is 2*pi
Wc = Wc1 / (0.5 * fs)
```

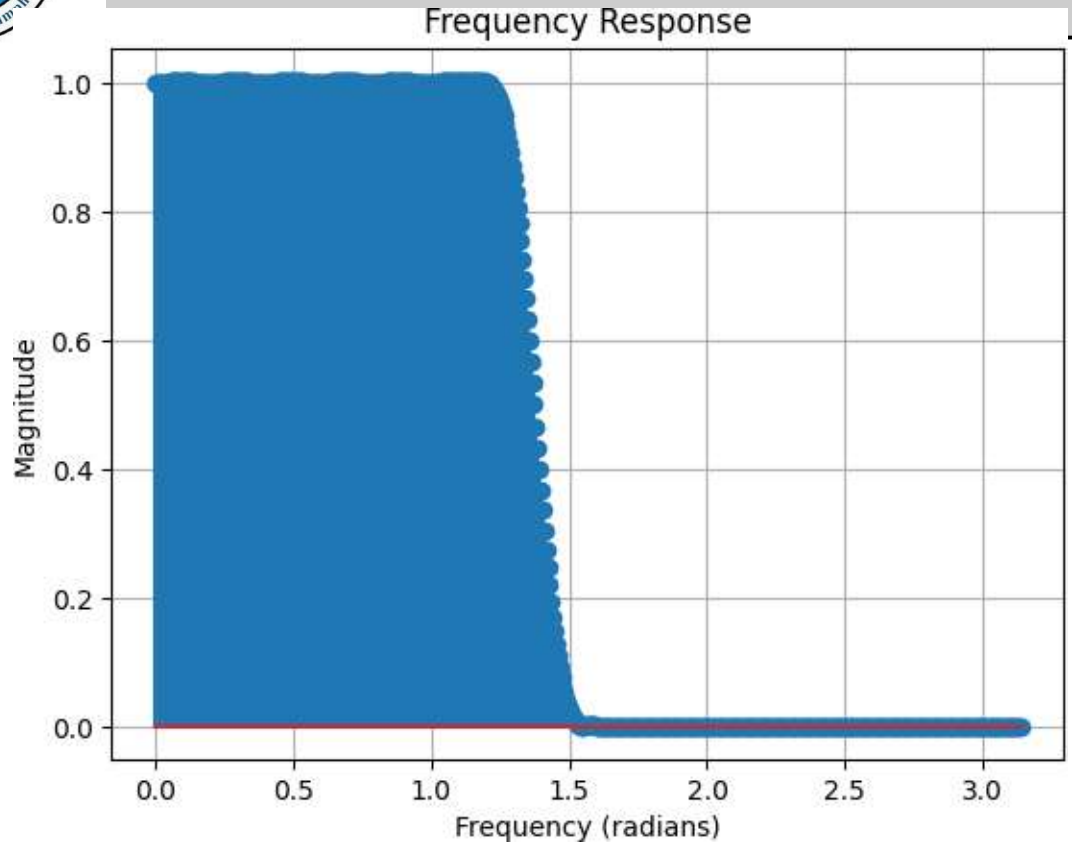
```
print('Cutoff frequency =', Wc)

# Calculate the impulse response of the filter
h = firwin(N, Wc, window='hamming')

print('Impulse Response of FIR filter:')
print(h)

# Plot the frequency response
plt.figure(1)
w, H = freqz(h)
plt.stem(w, np.abs(H))
plt.title('Frequency Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Magnitude')
plt.grid(True)

# Plot the impulse response
plt.figure(2)
n = np.arange(N)
plt.stem(n, h)
plt.xlabel('n')
plt.ylabel('h(n)')
plt.title('Impulse Response of Filter')
plt.grid(True)

plt.show()
```

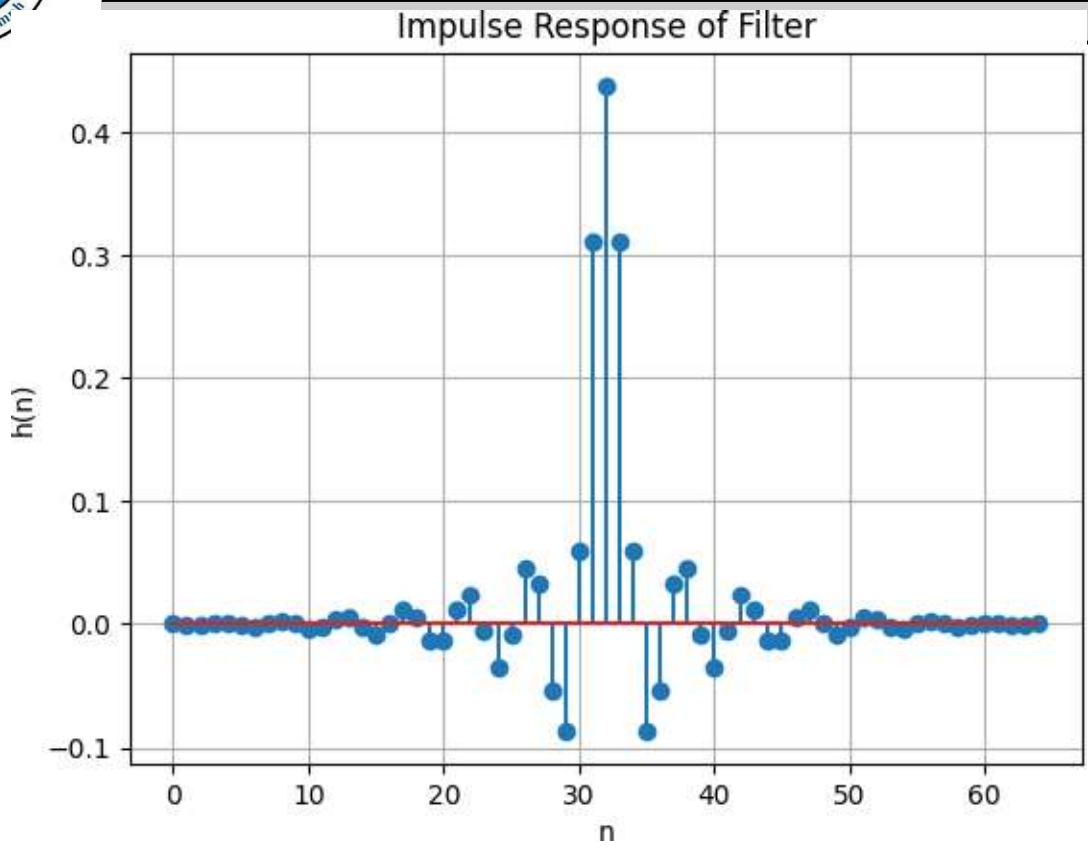
```
subplot(2,1,2);
plot(W/pi,angle(H));
title('phase response of lpf');
ylabel('angle ---- >');
xlabel('normalized frequency ----->');
```

Output Wave forms:

```
Order of the FIR filter N = 65
Cutoff frequency = 0.4375
Impulse Response of FIR filter:
[-1.36563720e-18 -8.28743024e-04 -3.61058348e-04  9.11733196e-04
  9.25421244e-04 -8.80562332e-04 -1.78338501e-03  4.58510044e-04
  2.85062249e-03  6.70701873e-04 -3.80571494e-03 -2.72389497e-03
  4.09988631e-03  5.66730655e-03 -3.04989910e-03 -9.09723779e-03
  9.21805112e-18  1.21887797e-02  5.48441633e-03 -1.37251478e-02
 -1.34429342e-02  1.21789049e-02  2.34177943e-02 -5.74486057e-03
 -3.44601894e-02 -7.95244816e-03  4.52559436e-02  3.34789130e-02
 -5.43502855e-02 -8.65551612e-02 6.04241311e-02 3.11793705e-01
  4.37909506e-01  3.11793705e-01  6.04241311e-02 -8.65551612e-02
 -5.43502855e-02  3.34789130e-02  4.52559436e-02 -7.95244816e-03
 -3.44601894e-02 -5.74486057e-03  2.34177943e-02  1.21789049e-02
 -1.34429342e-02 -1.37251478e-02 5.48441633e-03 1.21887797e-02
  9.21805112e-18 -9.09723779e-03 -3.04989910e-03  5.66730655e-03
  4.09988631e-03 -2.72389497e-03 -3.80571494e-03  6.70701873e-04
  2.85062249e-03  4.58510044e-04 -1.78338501e-03 -8.80562332e-04
  9.25421244e-04  9.11733196e-04 -3.61058348e-04 -8.28743024e-04
 -1.36563720e-18]
```

MRC

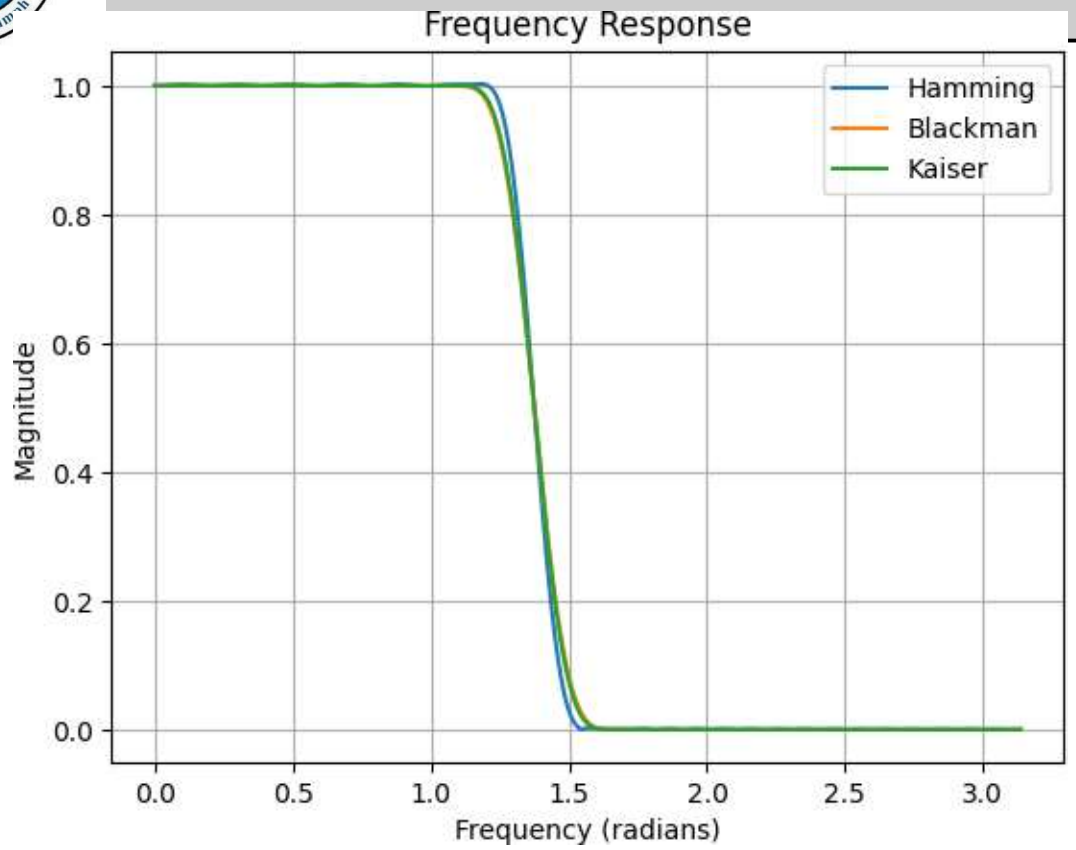

Frequency Response
Magnitude
Frequency (radians)
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

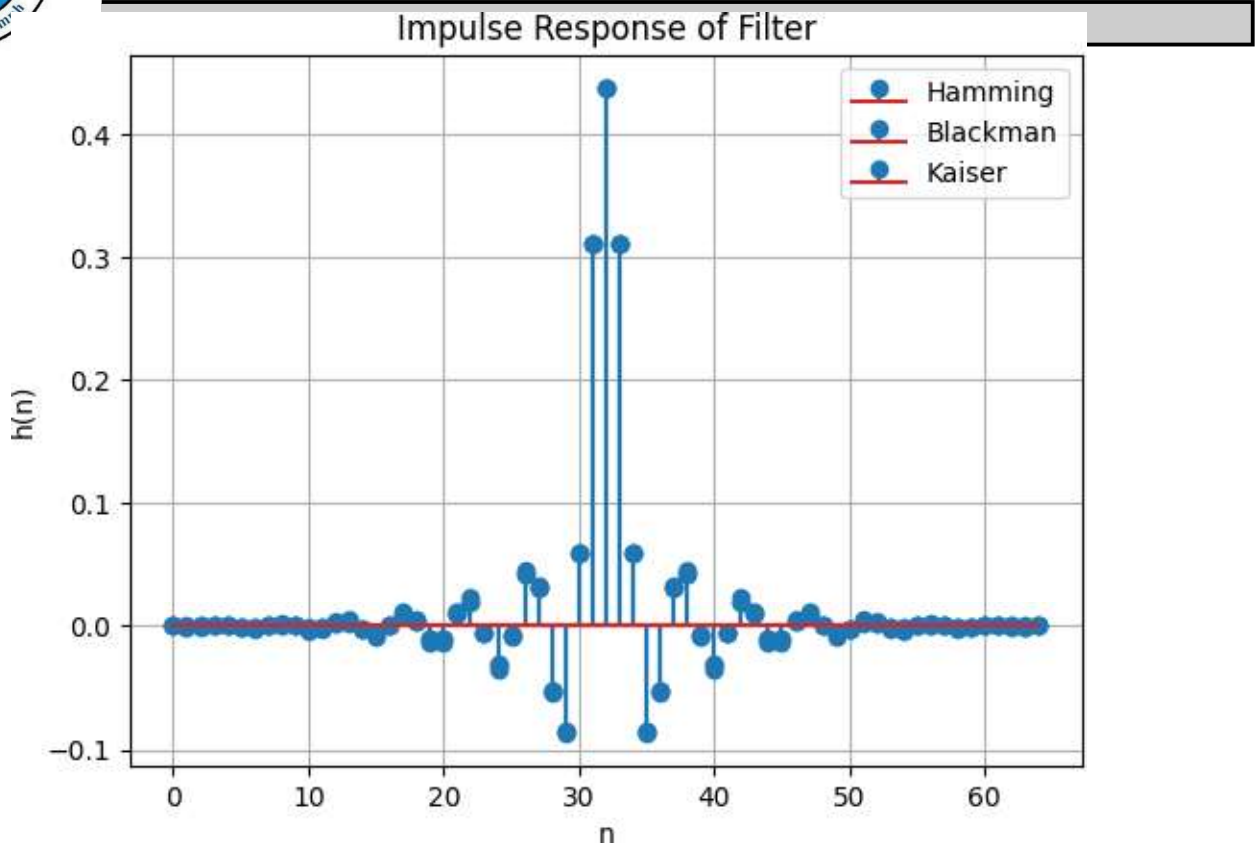


For different window methods

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, freqz

# Input specifications
wp = 0.375 * np.pi
ws = 0.5 * np.pi
wt = ws - wp

# Calculate the order of the filter
n1 = np.ceil(8 * np.pi / wt)
N = int(n1 + (n1 - 1) % 2)

print('Order of the FIR filter N =', N)

# Calculate the filter coefficients using different window functions

# Hamming window
h_hamming = firwin(N, Wc, window='hamming')

# Blackman window
h_blackman = firwin(N, Wc, window='blackman')
```

```
# Kaiser window
beta = 8 # Beta parameter for the Kaiser window
h_kaiser = firwin(N, Wc, window=('kaiser', beta))

# Plot the frequency response for different window functions
plt.figure(1)
w, H_hamming = freqz(h_hamming)
w, H_blackman = freqz(h_blackman)
w, H_kaiser = freqz(h_kaiser)
plt.plot(w, np.abs(H_hamming), label='Hamming')
plt.plot(w, np.abs(H_blackman), label='Blackman')
plt.plot(w, np.abs(H_kaiser), label='Kaiser')
plt.title('Frequency Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Magnitude')
plt.legend()
plt.grid(True)

# Plot the impulse response for different window functions
plt.figure(2)
n = np.arange(N)
plt.stem(n, h_hamming, label='Hamming')
plt.stem(n, h_blackman, label='Blackman')
plt.stem(n, h_kaiser, label='Kaiser')
plt.xlabel('n')
plt.ylabel('h(n)')
plt.title('Impulse Response of Filter')
plt.legend()
plt.grid(True)

plt.show()
```


Frequency Response
Hamming
Blackman
Kaiser
Magnitude
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Frequency (radians)



Lab Task 1: Hamming Window

Write a Python program to design an FIR filter using the Hamming window. Perform the following steps:

Define the input specifications: wp = 0.375 * np.pi, ws = 0.5 * np.pi.
Calculate the order of the filter using the formula N = int(np.ceil(8 * np.pi / (ws - wp)) + 1).
Calculate the cutoff frequency: Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi).
Calculate the filter coefficients using the Hamming window: h_hamming = firwin(N, Wc, window='hamming').
Plot the frequency response and impulse response for the Hamming window.
Display the plots.

Lab Task 2: Blackman Window

Write a Python program to design an FIR filter using the Blackman window. Perform the following steps:

Define the input specifications: wp = 0.375 * np.pi, ws = 0.5 * np.pi.
Calculate the order of the filter using the formula N = int(np.ceil(8 * np.pi / (ws - wp)) + 1).
Calculate the cutoff frequency: Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi).
Calculate the filter coefficients using the Blackman window: h_blackman = firwin(N, Wc, window='blackman').

Plot the frequency response and impulse response for the Blackman window.
Display the plots.

Lab Task 3: Kaiser Window

Write a Python program to design an FIR filter using the Kaiser window. Perform the following steps:

Define the input specifications: wp = 0.375 * np.pi, ws = 0.5 * np.pi.
Calculate the order of the filter using the formula N = int(np.ceil(8 * np.pi / (ws - wp)) + 1).
Calculate the cutoff frequency: Wc = (wp + (ws - wp) / 2) / (0.5 * np.pi).
Set the beta parameter for the Kaiser window: beta = 8.
Calculate the filter coefficients using the Kaiser window: h_kaiser = firwin(N, Wc, window=('kaiser', beta)).
Plot the frequency response and impulse response for the Kaiser window.
Display the plots

EXP. NO: 11

IMPLEMENTATIONS OF LP IIR FILTER

Objective: To implement LP IIR filter for a given sequence.

Software: Python

THEORY:

IIR filters are digital filters with infinite impulse response. Unlike FIR filters, they have the feedback (a recursive part of a filter) and are known as recursive digital filters therefore.

For this reason IIR filters have much better frequency response than FIR filters of the same order. Unlike FIR filters, their phase characteristic is not linear which can cause a problem to the systems which need phase linearity. For this reason, it is not preferable to use IIR filters in digital signal processing when the phase is of the essence. Otherwise, when the linear phase characteristic is not important, the use of IIR filters is an excellent solution.

There is one problem known as a potential instability that is typical of IIR filters only. FIR filters do not have such a problem as they do not have the feedback. For this reason,it is always necessary to check after the design process whether the resulting IIR filter is stable or not.

MRCE

IIR FILTER DESIGN

For the given specifications to Design a digital IIR filter, first we need to design analog filter (Butterworth or chebyshev). The resultant analog filter is transformed to digital filter by using either “Bilinear transformation or Impulse Invariant transformation”.

Algorithm:

Step I : Enter the pass band ripple (rp) and stop band ripple (rs).

Step II : Enter the pass band frequency (wp) and stop band frequency (ws).

Step III : Get the sampling frequency (fs).

Step IV : Calculate normalized pass band frequency, and normalized stop band frequency w1 and w2 respectively. w1 = 2 * wp /fs w2 = 2 * ws /fs

Step V : Make use of the following function to calculate order of filter

Butterworth filter order

[n,wn]=buttord(w1,w2,rp,rs)

Chebyshev filter order

[n,wn]=cheb1ord(w1,w2,rp,rs)

Step VI : Design an nth order digital low pass Butterworth or Chebyshev filter using the following statements.

Butterworth filter

[b, a]=butter (n, wn)

Step VII : Find the digital frequency response of the filter by using 'freqz( )' function

Step VIII : Calculate the magnitude of the frequency response in decibels (dB)

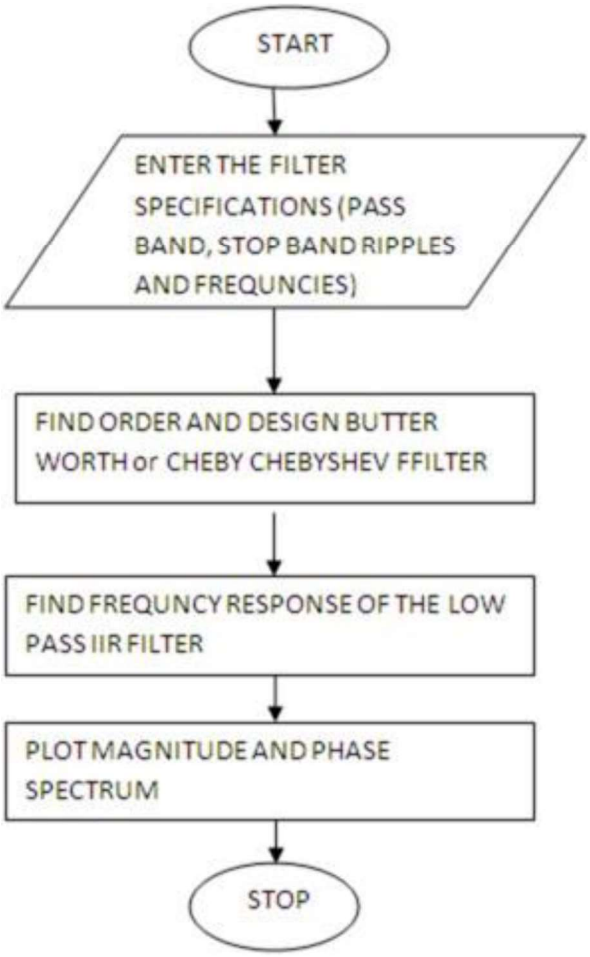
mag=20*log10 (abs (H))

Step IX : Plot the magnitude response [magnitude in dB Vs normalized frequency]

Step X : Calculate the phase response using angle (H)

Step XI : Plot the phase response [phase in radians Vs normalized frequency (Hz)].

Flow Chart:


START
ENTER THE FILTER SPECIFICATIONS (PASS BAND, STOP BAND RIPPLES AND FREQUNCIES)
FIND ORDER AND DESIGN BUTTER WORTH or CHEBY CHEBYSHEV FFILTER
FIND FREQUNCY RESPONSE OF THE LOW PASS IIR FILTER
PLOT MAGNITUDE AND PHASE SPECTRUM
STOP

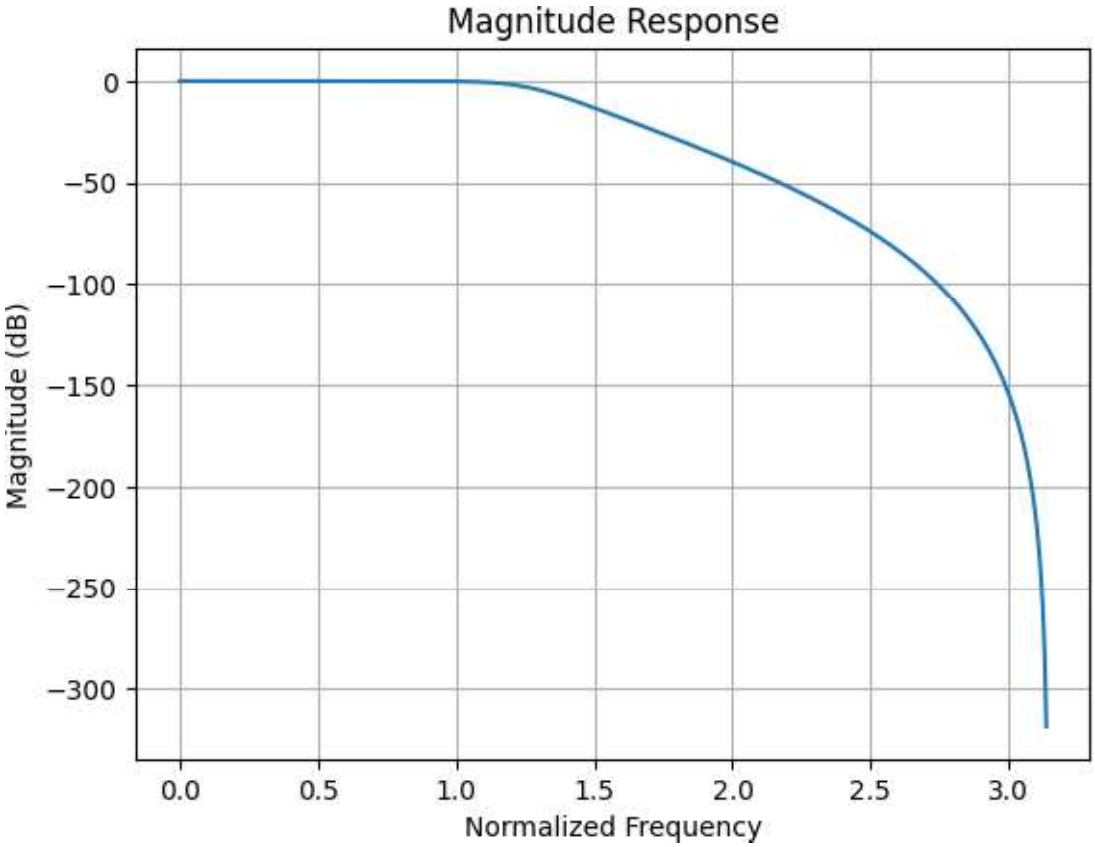
MRCE

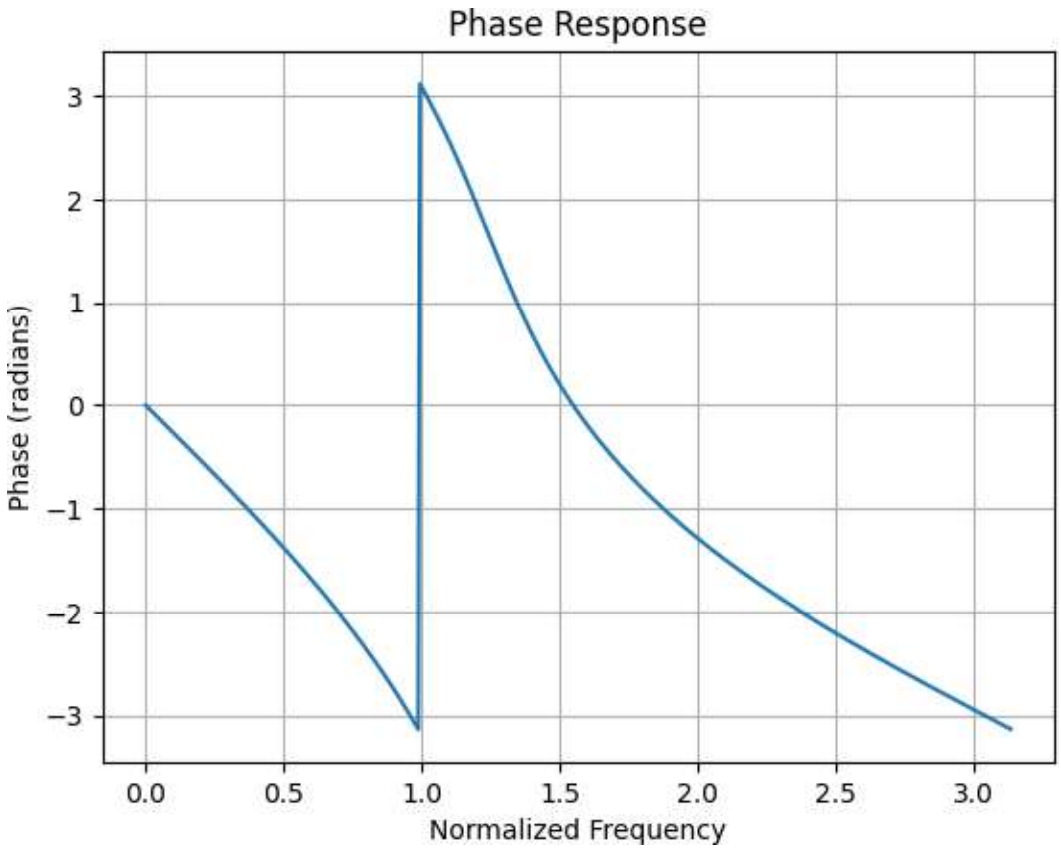
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, cheby1, freqz
from scipy.constants import pi

# Step I: Enter the pass band ripple (rp) and stop band ripple (rs)
rp = 0.1
rs = 40

# Step II: Enter the pass band frequency (wp) and stop band frequency (ws)
wp = 0.375 * pi
ws = 0.5 * pi

# Step III: Get the sampling frequency (fs)
fs = 2 * pi

# Step IV: Calculate normalized pass band frequency, and normalized stop band frequency w1 and
w2 respectively
w1 = 2 * wp / fs
w2 = 2 * ws / fs

# Step V: Calculate the filter order
# Butterworth filter order
# [n, wn] = buttord(w1, w2, rp, rs)

# Chebyshev filter order
# [n, wn] = cheb1ord(w1, w2, rp, rs)

# Example filter order and wn values for demonstration
n = 6
wn = 0.4
```

```
# Step VI: Design an nth order digital low pass Butterworth or Chebyshev filter
# Butterworth filter
b, a = butter(n, wn)

# Chebyshev filter
# b, a = cheby1(n, rp, wn)

# Step VII: Find the digital frequency response of the filter
w, H = freqz(b, a)

# Step VIII: Calculate the magnitude of the frequency response in decibels (dB)
mag = 20 * np.log10(abs(H))

# Step IX: Plot the magnitude response [magnitude in dB Vs normalized frequency]
plt.figure(1)
plt.plot(w, mag)
plt.title('Magnitude Response')
plt.xlabel('Normalized Frequency')
plt.ylabel('Magnitude (dB)')
plt.grid(True)

# Step X: Calculate the phase response using angle(H)
phase = np.angle(H)

# Step XI: Plot the phase response [phase in radians Vs normalized frequency (Hz)]
plt.figure(2)
plt.plot(w, phase)
plt.title('Phase Response')
plt.xlabel('Normalized Frequency')
plt.ylabel('Phase (radians)')
plt.grid(True)

plt.show()
```

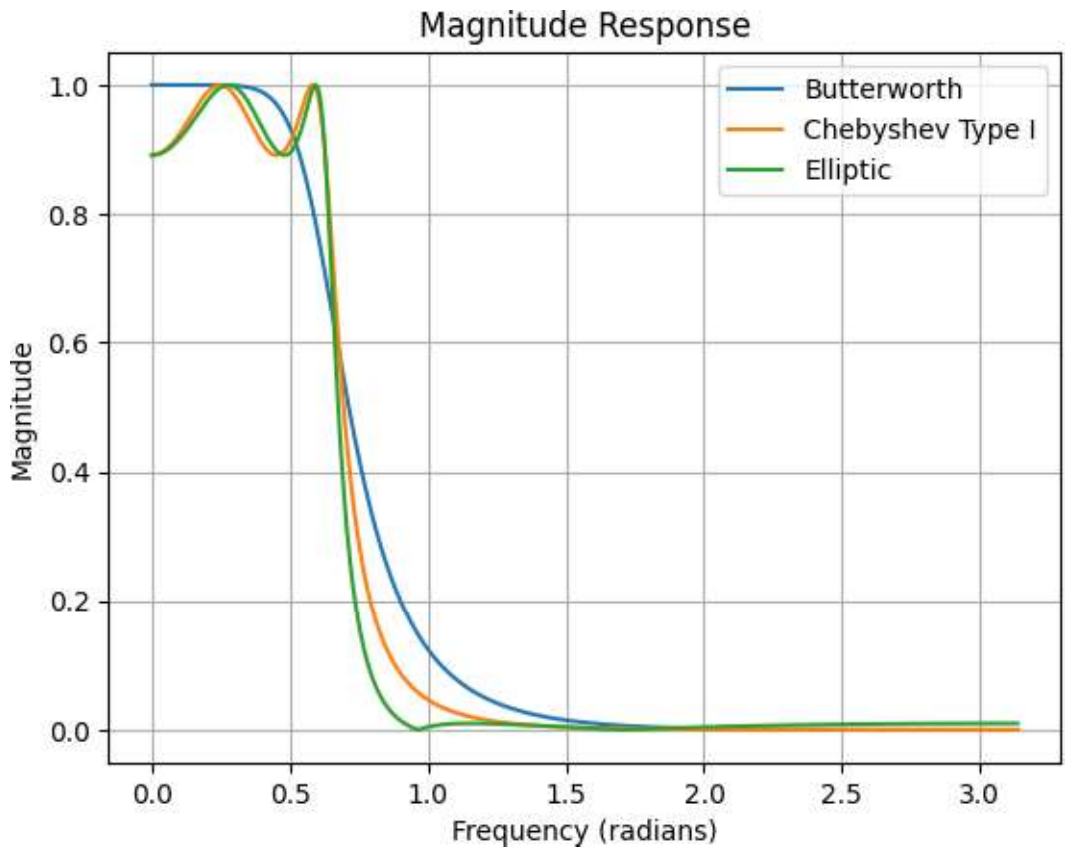
Output waveforms:

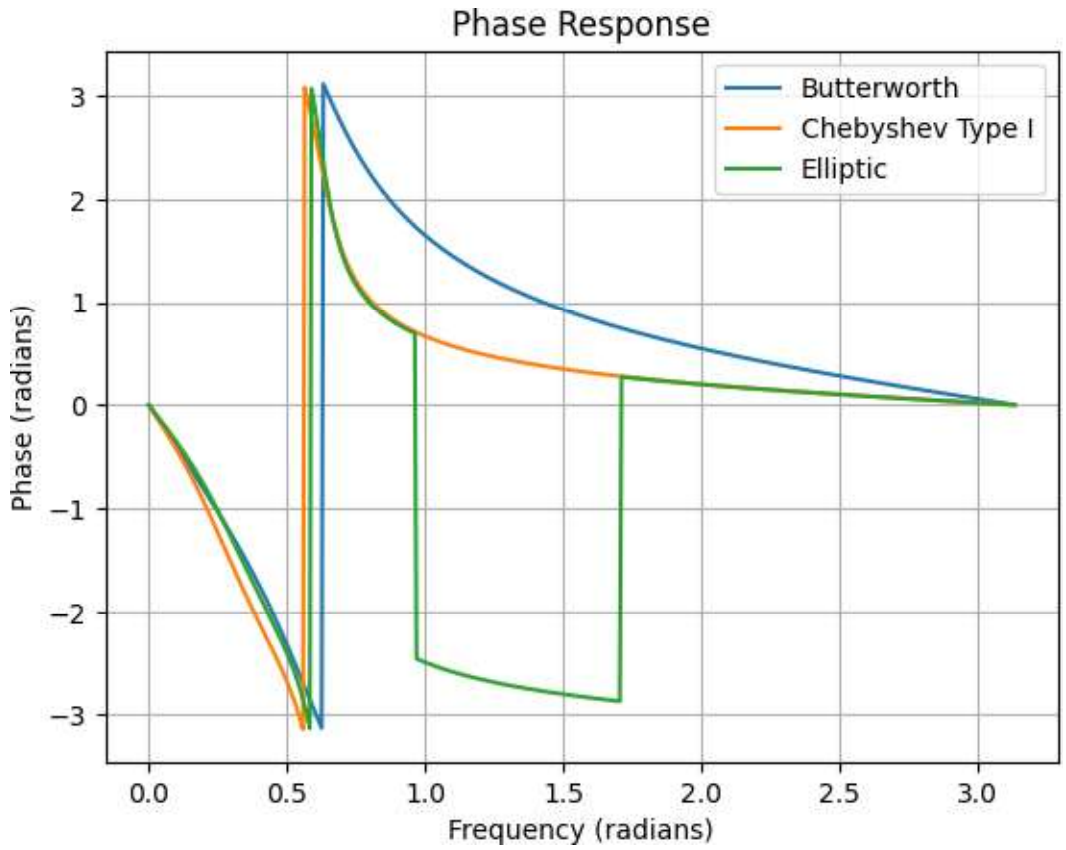





```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, cheby1, ellip, freqz

# Filter specifications
order = 4
cutoff_freq = 0.2  # Normalized cutoff frequency

# Design the Butterworth filter
b_butter, a_butter = butter(order, cutoff_freq, btype='low', analog=False, output='ba')

# Design the Chebyshev Type I filter
rp_cheby1 = 1  # Passband ripple in dB
b_cheby1, a_cheby1 = cheby1(order, rp_cheby1, cutoff_freq, btype='low', analog=False, output='ba')

# Design the elliptic filter
rp_ellip = 1 # Passband ripple in dB
rs_ellip = 40  # Stopband attenuation in dB
b_ellip, a_ellip = ellip(order, rp_ellip, rs_ellip, cutoff_freq, btype='low', analog=False, output='ba')

# Compute the frequency response for each filter
w, h_butter = freqz(b_butter, a_butter)
w, h_cheby1 = freqz(b_cheby1, a_cheby1)
```

```
w, h_ellip = freqz(b_ellip, a_ellip)

# Plot the magnitude response for each filter
plt.figure()
plt.plot(w, np.abs(h_butter), label='Butterworth')
plt.plot(w, np.abs(h_cheby1), label='Chebyshev Type I')
plt.plot(w, np.abs(h_ellip), label='Elliptic')
plt.title('Magnitude Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Magnitude')
plt.legend()
plt.grid(True)

# Plot the phase response for each filter
plt.figure()
plt.plot(w, np.angle(h_butter), label='Butterworth')
plt.plot(w, np.angle(h_cheby1), label='Chebyshev Type I')
plt.plot(w, np.angle(h_ellip), label='Elliptic')
plt.title('Phase Response')
plt.xlabel('Frequency (radians)')
plt.ylabel('Phase (radians)')
plt.legend()
plt.grid(True)

plt.show()
```





Lab Task: Implementation of IIR Filters

Design three different IIR filters using the given specifications:

Filter order: 4
Cutoff frequency: 0.2 (normalized frequency)
Algorithm 1: Butterworth Filter

Use the butter function from SciPy to design a Butterworth filter with the given order and cutoff frequency.
Obtain the filter coefficients b_butter and a_butter.
Algorithm 2: Chebyshev Type I Filter

Set the passband ripple to 1 dB.
Use the cheby1 function from SciPy to design a Chebyshev Type I filter with the given order, passband ripple, and cutoff frequency.
Obtain the filter coefficients b_cheby1 and a_cheby1.

Algorithm 3: Elliptic Filter

Set the passband ripple to 1 dB and the stopband attenuation to 40 dB.
Use the ellip function from SciPy to design an elliptic filter with the given order, passband ripple, stopband attenuation, and cutoff frequency.
Obtain the filter coefficients b_ellip and a_ellip.
Plot the magnitude response for each filter:

Compute the frequency response of each filter using the freqz function from SciPy.
Plot the magnitude response of the Butterworth filter using plt.plot(w, np.abs(h_butter), label='Butterworth').
Plot the magnitude response of the Chebyshev Type I filter using plt.plot(w, np.abs(h_cheby1), label='Chebyshev Type I').
Plot the magnitude response of the elliptic filter using plt.plot(w, np.abs(h_ellip), label='Elliptic').
Set the title to 'Magnitude Response' using plt.title('Magnitude Response').
Set the x-axis label to 'Frequency (radians)' using plt.xlabel('Frequency (radians)').
Set the y-axis label to 'Magnitude' using plt.ylabel('Magnitude').
Display the legend using plt.legend().
Enable grid lines using plt.grid(True).
Plot the phase response for each filter:

Plot the phase response of the Butterworth filter using plt.plot(w, np.angle(h_butter), label='Butterworth').
Plot the phase response of the Chebyshev Type I filter using plt.plot(w, np.angle(h_cheby1), label='Chebyshev Type I').
Plot the phase response of the elliptic filter using plt.plot(w, np.angle(h_ellip), label='Elliptic').
Set the title to 'Phase Response' using plt.title('Phase Response').
Set the x-axis label to 'Frequency (radians)' using plt.xlabel('Frequency (radians)').
Set the y-axis label to 'Phase (radians)' using plt.ylabel('Phase (radians)').
Display the legend using plt.legend().
Enable grid lines using plt.grid(True).
Display the plots using plt.show().